BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Trương Quốc Lê Huy
MSSV / Lớp	3123410132

Mục tiêu học tập

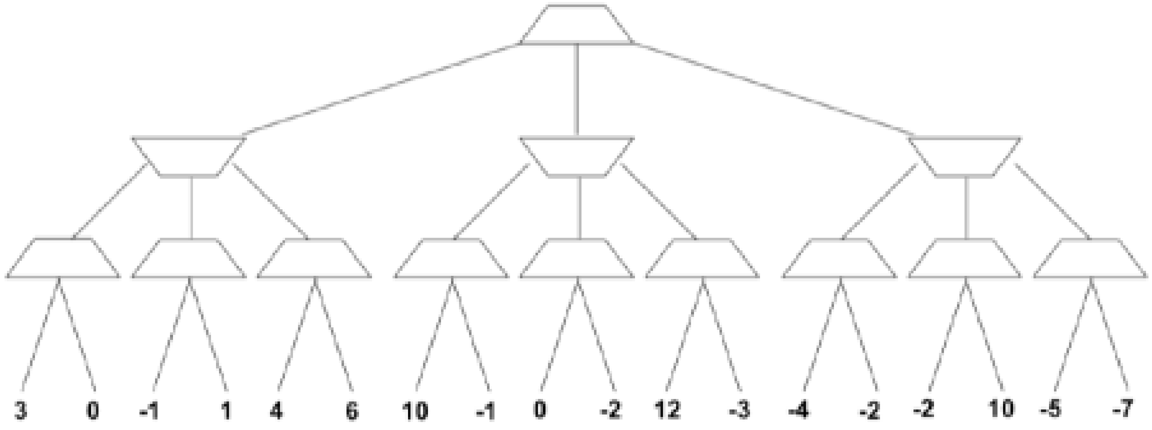
• Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa. • Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.

• Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP. • Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi. • Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Quy tắc:

- MAX: chọn giá trị lớn nhất.
- MIN: chọn giá trị nhỏ nhất.

- Tính từ các nút lá lên nút gốc.

Bước 1: Tính các nút MAX ở tầng dưới cùng

Nút	**Các giá trị lá**	**Giá trị MAX**
A1	3, 0	3
A2	-1, 1	1
A3	4, 6	6
B1	10, 0	10
B2	0, -2	0
B3	12, -3	12
C1	-4, -2	-2
C2	-2, 10	10
C3	-5, -7	-5

Bước 2: Tính các nút MIN ở tầng giữa

- Nhánh trái: $\min(3, 1, 6) = 1$
- Nhánh giữa: $\min(10, 0, 12) = 0$
- Nhánh phải: $\min(-2, 10, -5) =- 5$

Bước 3: Tính nút gốc MAX

$$\max(1, 0, -5) = 1$$

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

MAX nên chọn nhánh bên trái vì có giá trị Minimax lớn nhất.

Giá trị trò chơi:

$$\boxed{V = 1}$$

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Khởi tạo tại nút gốc: $\alpha = -\infty, \beta = +\infty$.

1. Xét nhánh trái (MIN)

- Nút MAX thứ nhất: $\max(3, 0) = 3$.
- Nút MAX thứ hai: $\max(-1, 1) = 1$.
- Nút MAX thứ ba: $\max(4, 6) = 6$.
- Nút MIN trái: $\min(3, 1, 6) = 1$.

Cập nhật tại gốc: $\alpha = 1$.

2. Xét nhánh giữa (MIN)

- Nút MAX thứ nhất: $\max(10, 0) = 10$.
- Nút MAX thứ hai: lá đầu tiên có giá trị 0. Khi đó $\beta = 0$ tại nút MIN giữa.

Vì $\beta = 0 \leq \alpha = 1$, cắt tỉa lá -2 và toàn bộ nhánh MAX thứ ba có các lá $12, -3$.

Giá trị nút MIN giữa là 0.

3. Xét nhánh phải (MIN)

- Nút MAX thứ nhất: $\max(-4, -2) = -2$.
- Tại nút MIN phải: $\beta = -2 \leq \alpha = 1$.

Do đó, cắt tỉa toàn bộ hai nút MAX còn lại, gồm các lá $(-2, 10)$ và $(-5, -7)$.

Giá trị nút MIN phải không cần tính chính xác vì MAX tại gốc đã có lựa chọn tốt hơn.

d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.
Các nút lá bị cắt tỉa:

- Nhánh giữa: $-2, 12, -3$.
- Nhánh phải: $-2, 10, -5, -7$.

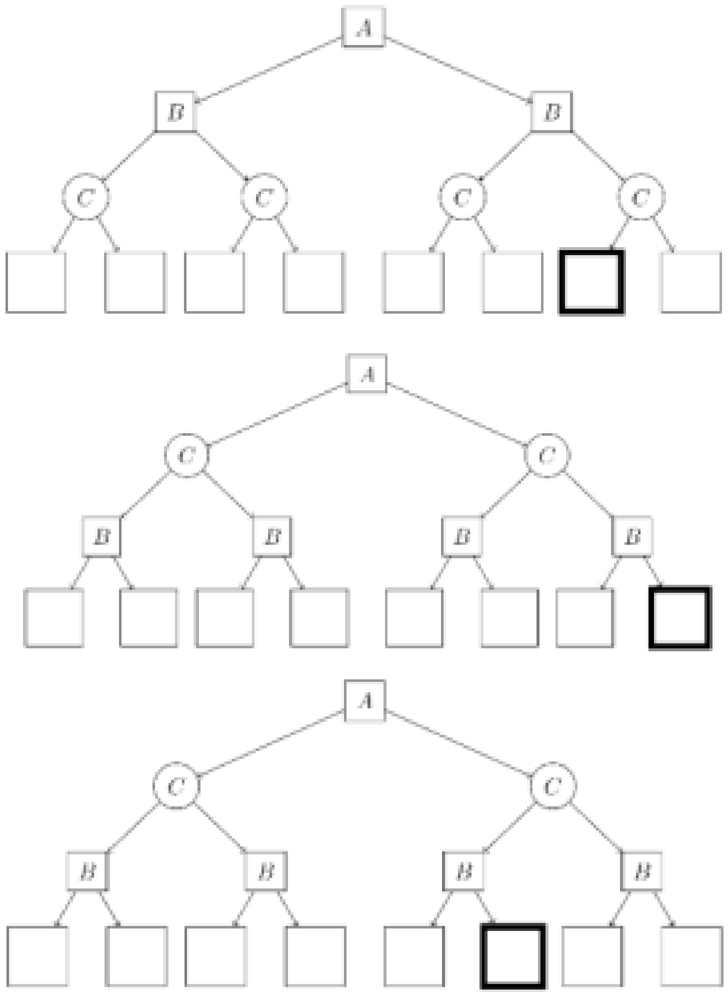
Tổng cộng:

7 nút lá bị cắt tỉa

Phương pháp	**Số nút lá được thăm**	**Số nút lá không thăm**
Minimax	18	0
Alpha-Beta	11	7

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi “Không thể” và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Cây có cấu trúc $A \rightarrow B \rightarrow C$.

Duyệt từ trái sang phải:

1. Tại nút C bên trái, tính trung bình giá trị các nút lá.
2. Tại nút C tiếp theo, tiếp tục tính giá trị kỳ vọng.
3. Nút B lấy giá trị nhỏ nhất trong các nút C con.
4. Thực hiện tương tự với nhánh phải.
5. Nút A chọn giá trị lớn nhất giữa hai nút B.

Điều kiện cắt tỉa: nếu cận trên của giá trị kỳ vọng tại một nhánh không vượt quá giá trị tốt nhất mà MAX đã có, nhánh đó có thể bị cắt. Tuy nhiên, phải biết các giới hạn giá trị lá và xác suất để kết luận chính xác.

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Cấu trúc cây: $A \rightarrow C \rightarrow B$.

1. Tại mỗi nút B, lấy giá trị nhỏ nhất của hai nút lá.
2. Tại mỗi nút C, tính trung bình các giá trị B con.
3. Nút A chọn giá trị lớn nhất giữa hai nút C.

Điều kiện cắt tỉa: khi đã có đủ thông tin để xác định cận trên của một nút C không thể vượt qua giá trị tốt nhất hiện tại của A, có thể cắt phần còn lại của nhánh đó.

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Cấu trúc cây: $A \rightarrow C \rightarrow B$, với thứ tự duyệt từ trái sang phải.

1. Tính giá trị các nút B từ các nút lá.
2. Tính giá trị kỳ vọng tại nút C bên trái.
3. Duyệt nút C bên phải và các nút B con theo thứ tự trái sang phải.
4. Tại nút A, chọn giá trị kỳ vọng lớn nhất.

Điều kiện cắt tỉa: nếu tổng giá trị tối đa có thể đạt được từ phần chưa duyệt của nút C không đủ để vượt giá trị tốt nhất hiện có tại A, có thể cắt phần còn lại.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

- Minimax: tại nút MAX hoặc MIN, có thể dùng α và β để xác định một nhánh chắc chắn không ảnh hưởng đến kết quả tối ưu. Vì vậy, có thể cắt nhánh ngay khi điều kiện $\alpha \geq \beta$ thỏa mãn.
- Expectimax: tại nút Chance, giá trị được tính theo kỳ vọng có trọng số:

$$V(C) = \sum_i P_i V_i$$

Giá trị của nút Chance phụ thuộc vào tất cả kết quả có thể xảy ra. Vì thế, không thể áp dụng trực tiếp điều kiện cắt tỉa Minimax nếu chưa có cận trên, cận dưới và xác suất phù hợp.

Kết luận: Alpha-Beta cắt tỉa dựa trên quan hệ giữa MAX và MIN; còn trong Expectimax, việc cắt tỉa phải dựa trên các cận của giá trị kỳ vọng và xác suất của những kết quả chưa xét.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$. . ☑ Đúng ☐ Sai

Giải thích: Minimax giả định đối thủ luôn chọn nước đi bất lợi nhất cho MAX, còn Expectimax xét giá trị kỳ vọng theo phân phối xác suất đã biết. Giá trị kỳ vọng không nhỏ hơn giá trị khi đối thủ luôn chọn kết quả thấp nhất, nếu các giả định về trạng thái, hành động và giá trị lá là nhất quán.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M . ☑ Đúng ☐ Sai

Giải thích: Chính sách Minimax bảo đảm giá trị tối thiểu V_M trước đối thủ có thể chọn kết quả bất lợi nhất. Khi thay đối thủ bằng nút ngẫu nhiên có các xác suất đã biết, giá trị kỳ vọng của chính sách này không thấp hơn giá trị trong trường hợp luôn gặp kết quả xấu nhất.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất

là V_E. ☐ Đúng ☑ Sai

Giải thích: Chính sách Minimax tối ưu cho tình huống đối thủ chơi đối kháng, không nhất thiết tối ưu cho tình huống ngẫu nhiên. Expectimax có thể chọn một chính sách khác đem lại giá trị kỳ vọng cao hơn. Vì vậy, chính sách Minimax không bảo đảm đạt ít nhất V_E. Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi

thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết. ☐ Minimax với tiện ích trạng thái cuối +1/−1

☑ Expectimax với tiện ích trạng thái cuối +1/−1

☐ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua ☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

Expectimax phù hợp khi kết quả hành động chịu ảnh hưởng của yếu tố ngẫu nhiên có phân phối xác suất xác định. Tiện ích $+\ 1$ khi thắng và $-\ 1$ khi thua phản ánh mục tiêu tối đa hóa xác suất thắng trừ xác suất thua.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).

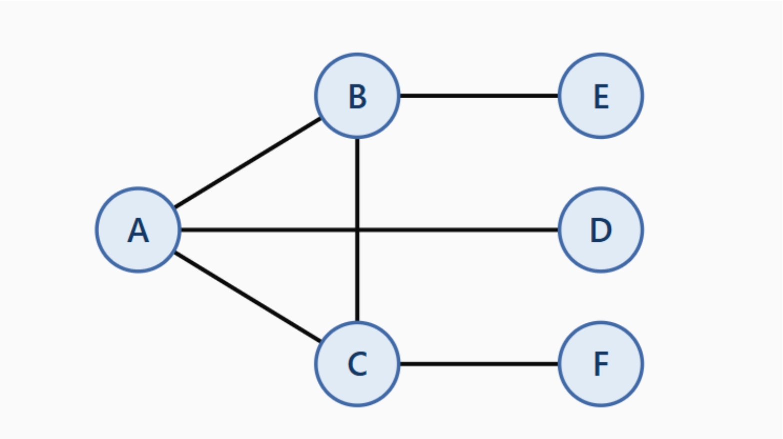
Có 6 biến A, B, C, D, E, F. Mỗi biến là một đỉnh, mỗi ràng buộc nhị phân là một cạnh.

Tập đỉnh:

$$V = \{A, B, C, D, E, F\}$$

Tập cạnh:

$$E = \{(A, B), (A, C), (A, D), (B, C), (B, E), (C, F)\}$$



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Đáp án: B, C, D.

Giải thích: Forward Checking chỉ cập nhật miền của các biến chưa gán có liên kết trực tiếp với biến vừa gán.

Các biến kề với A là B, C, D, nên miền của ba biến này có thể bị loại bỏ giá trị không tương thích với giá trị đã gán cho A.

Các biến E, F không kề trực tiếp với A, nên Forward Checking chưa cập nhật miền của chúng ở bước này.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

Đáp án: C, E.

Giải thích: Các biến kề với B là A, C, E. Vì A đã được gán giá trị và B vừa được gán giá trị, Forward Checking cập nhật miền của các biến chưa gán kề với B, gồm:

- C: do có ràng buộc (B, C).
- E: do có ràng buộc (B, E).

Miền của D, F không được cập nhật trực tiếp bởi lần gán B.

d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

Đáp án: B, C, D, E, F.

Giải thích: Arc Consistency có thể lan truyền ảnh hưởng qua nhiều cạnh:

- B, C, D bị ảnh hưởng trực tiếp bởi các ràng buộc với A.
- Khi miền B thay đổi, miền E có thể thay đổi do ràng buộc (B, E).
- Khi miền C thay đổi, miền F có thể thay đổi do ràng buộc (C, F).
- Ràng buộc (B, C) cũng có thể khiến miền của B, C tiếp tục thu hẹp.

Việc miền có thực sự thay đổi hay không phụ thuộc vào các ràng buộc và miền giá trị cụ thể.

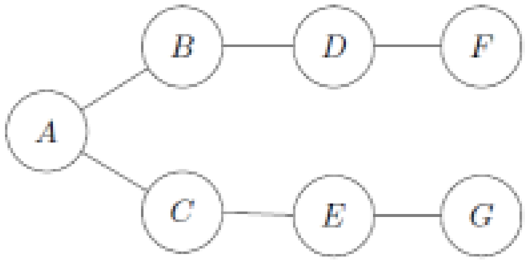
e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào
có thể thay đổi?

Đáp án: C, D, E, F.

Giải thích: A, B đã được gán nên không còn là các biến có miền chưa gán cần xét. Sau khi gán B, Arc Consistency có thể tiếp tục lan truyền:

- C: qua ràng buộc (B, C).
- D: qua ràng buộc (A, D), nếu miền của A hoặc các ràng buộc liên quan đã gây ra thay đổi.
- E: qua ràng buộc (B, E).
- F: qua ràng buộc (C, F), nếu miền của C thay đổi.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi
biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G

Gán A, B rồi kiểm tra; sau đó gán C, D, rồi E, F. Các lần kiểm tra có thể phát hiện mâu thuẫn sau khi một số biến đã được gán.

g) Thứ tự G → E → C → A → B → D → F

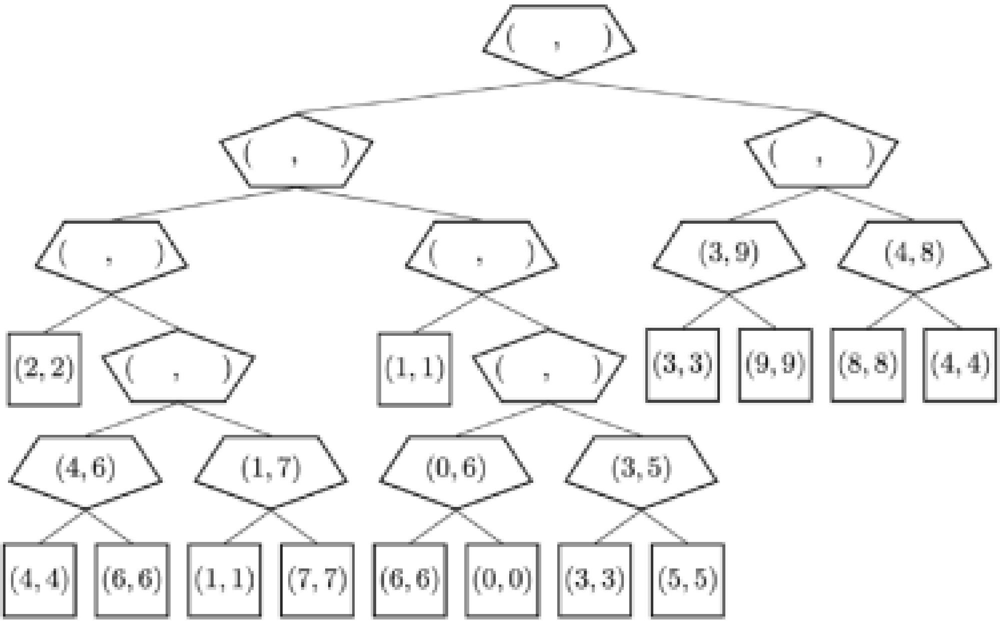
Gán từ đầu nhánh dưới về nút nối A, sau đó chuyển sang nhánh trên. Các ràng buộc trong nhánh dưới được kiểm tra sau mỗi hai biến.

h) Thứ tự G → F → E → D → C → B → A

Gán xen kẽ hai nhánh, có thể khiến các biến ở hai nhánh được gán trước khi nút nối *A* được gán.

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper. Để điền đúng, cần xác định quy tắc chọn của từng loại nút. Theo mô hình Pacman–Casper thường dùng, Pacman chọn nhánh tối đa hóa u, còn Casper chọn nhánh dựa trên v, thể hiện mức độ thân thiện thực tế. Các giá trị ở nút cha được tổng hợp từ các nút con theo quy tắc tương ứng.

Tính từ dưới lên:

- Nút Casper có hai lá $(4, 4)$, $(6, 6)$ cho giá trị $(4, 6)$.
- Nút Casper có hai lá $(1, 1)$, $(7, 7)$ cho giá trị $(1, 7)$.
- Nút Casper có hai lá $(6, 6)$, $(0, 0)$ cho giá trị $(0, 6)$.
- Nút Casper có hai lá $(3, 3)$, $(5, 5)$ cho giá trị $(3, 5)$.

Tại các nút Pacman, chọn nhánh có u lớn hơn. Tại các nút Casper, u là giá trị nhỏ nhất mà Pacman dự đoán, còn v là giá trị lớn nhất theo hành vi thân thiện thực tế.

Suy ra các nút còn thiếu, từ dưới lên:

Vị trí nút	**Giá trị** (u, v)
Pacman phía trái, giữa $(4, 6)$ và $(1, 7)$	$(4, 6)$
Pacman phía trái, giữa $(0, 6)$ và $(3, 5)$	$(3, 5)$
Casper phía trái, giữa $(2, 2)$ và $(4, 6)$	$(2, 6)$
Casper phía trái, giữa $(1, 1)$ và $(3, 5)$	$(1, 5)$
Casper ở nhánh trái của gốc	$(1, 6)$
Casper ở nhánh phải của gốc	$(3, 9)$
Nút gốc	$(3, 9)$

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi. Duyệt từ trái sang phải. Sau khi tính nhánh con bên trái của nút Pacman phía phải trong cây con bên trái, ta có $\alpha = 1$. Tại nút Casper kế tiếp, nhánh đầu cho $u = 0$, nên $\beta = 0$.

Vì $\beta \le \alpha$, có thể cắt nhánh $(3, 5)$ cùng toàn bộ các lá bên dưới nó khi chỉ cần xác định giá trị mà Pacman dự đoán u.

Lưu ý: Không được dùng phép cắt này để bỏ qua việc tính v nếu cần xác định đầy đủ hành vi thực tế của Casper.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Khi Pacman biết Casper thân thiện, cả hai đều muốn tối đa hóa tiện ích. Casper không còn bị Pacman dự đoán là chọn nhánh có u thấp nhất.

Vì thế, các nút Casper chuyển từ lựa chọn theo giá trị nhỏ nhất sang lựa chọn theo giá trị lớn nhất. Pacman sẽ chọn nhánh có tiện ích tốt nhất trong mô hình mới.

Nhánh bên phải có thể đạt tiện ích 9, lớn hơn mức tốt nhất 7 ở nhánh bên trái. Do đó, giá trị tại gốc trở thành:

$$(u,v)=(9,9)$$

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	**Điều kiện đầy đủ (Complete)**	**Điều kiện tối ưu (Optimal)**
Breadth-First Search (BFS)	Có, nếu hệ số phân nhánh *b* hữu hạn.	Có, nếu chi phí mỗi bước bằng nhau.
Depth-First Search (DFS)	Không, nếu không gian tìm kiếm có độ sâu vô hạn hoặc có vòng lặp. Có nếu không gian hữu hạn và được xử lý vòng lặp phù hợp.	Không, vì có thể tìm thấy nghiệm không có chi phí nhỏ nhất trước.
Iterative Deepening Search (IDS)	Có, nếu *b* hữu hạn và nghiệm nằm ở độ sâu hữu hạn.	Có, nếu chi phí mỗi bước bằng nhau.
A Search*	Có, nếu hệ số phân nhánh hữu hạn, chi phí bước có cận dưới dương và tồn tại nghiệm.	Có, nếu heuristic admissible; với đồ thị tìm kiếm có trạng thái lặp, cần heuristic consistent hoặc cơ chế mở lại nút phù hợp.

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận. BFS: Duyệt theo từng mức độ sâu nên tìm được nghiệm nông nhất. Nếu chi phí mỗi bước bằng nhau, nghiệm nông nhất cũng là nghiệm có chi phí thấp nhất.

DFS: Đi sâu theo một nhánh trước khi quay lui. Nếu gặp nhánh vô hạn hoặc vòng lặp, thuật toán có thể không tìm được nghiệm dù nghiệm tồn tại. DFS cũng không đảm bảo tìm nghiệm tối ưu.

IDS: Lặp lại DFS với giới hạn độ sâu tăng dần. Thuật toán kết hợp tính đầy đủ của BFS với mức sử dụng bộ nhớ thấp của DFS. Khi chi phí mỗi bước bằng nhau, IDS tìm được nghiệm nông nhất và tối ưu.

A* Search: Ưu tiên nút có giá trị:

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$: chi phí thực tế từ nút bắt đầu đến n.
- $h(n)$: chi phí ước lượng từ n đến đích.

Heuristic admissible không bao giờ ước lượng vượt chi phí thực tế tối ưu còn lại. Điều kiện này giúp A* đảm bảo tính tối ưu trong tìm kiếm cây; trong tìm kiếm đồ thị, heuristic consistent là điều kiện đủ phổ biến khi không mở lại các nút đã đóng.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng; cạnh $X \rightarrow Y$ thuộc đường tối ưu và đã được khám phá

Trả lời: Cần cập nhật chi phí cạnh và tính lại chi phí đường đi qua cạnh này. Đường đi cũ có thể không còn tối ưu. Nếu chi phí mới khiến một đường khác tốt hơn, A* cần cập nhật các giá trị g, f và xử lý lại những nút bị ảnh hưởng. Không nhất thiết chạy lại từ đầu.

b) Chi phí giảm; cạnh $X \rightarrow Y$ thuộc đường tối ưu và đã được khám phá

Trả lời: Cập nhật chi phí cạnh, giảm $g(Y)$ nếu đường qua X tốt hơn, sau đó cập nhật $f(Y)$ và lan truyền thay đổi đến các nút kế tiếp. Đường tối ưu có thể có chi phí thấp hơn. Nếu các nút liên quan đã đóng, có thể cần mở lại chúng.

c) Chi phí tăng; cạnh $X \rightarrow Y$ không thuộc đường tối ưu nhưng đã được khám phá

Trả lời: Nếu cạnh không thuộc đường tối ưu hiện tại và việc tăng chi phí không làm thay đổi các lựa chọn khác, đường tối ưu vẫn giữ nguyên. Tuy nhiên, cần kiểm tra xem cạnh này có nằm trên một đường thay thế có khả năng trở thành tối ưu hay không. Nếu không ảnh hưởng đến các đường đi tốt nhất, không cần tính lại toàn bộ.

d) Chi phí giảm; cạnh $X \rightarrow Y$ không thuộc đường tối ưu nhưng đã được khám phá

Trả lời: Cạnh này có thể tạo ra một đường đi mới tốt hơn đường tối ưu hiện tại. Cần cập nhật $g(Y)$, $f(Y)$ và lan truyền các cải thiện đến các nút tiếp theo. Nếu đường qua cạnh mới tốt hơn, phải cập nhật nghiệm. Không nhất thiết chạy lại từ đầu nếu thuật toán hỗ trợ cập nhật tăng dần.

e) Chi phí tăng; cạnh $X \rightarrow Y$ không thuộc đường tối ưu và chưa được khám phá

Trả lời: Nếu cạnh chưa được khám phá và không nằm trên đường tối ưu hiện tại, nghiệm hiện tại thường không bị ảnh hưởng. Khi A* khám phá cạnh này trong tương lai, thuật toán sẽ sử dụng chi phí mới. Không cần chạy lại từ đầu nếu không có ảnh hưởng nào khác.

f) Chi phí giảm; cạnh $X \rightarrow Y$ không thuộc đường tối ưu và chưa được khám phá

Trả lời: Cạnh này có thể tạo ra đường đi tốt hơn khi được khám phá. Nếu A* chưa xét cạnh, thuật toán sẽ sử dụng chi phí mới khi mở rộng X. Tuy nhiên, nếu A* đã kết thúc và cần bảo đảm nghiệm tối ưu theo đồ thị mới, phải tiếp tục tìm kiếm hoặc chạy lại thuật toán để kiểm tra khả năng tồn tại đường tốt hơn.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ: • A: act(percept) → f_A()

- B: act(percept) → f_B(percept)
- C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh

Đáp án: B.

Giải thích: Môi trường quan sát đầy đủ nên tác tử nhận được toàn bộ thông tin cần thiết về trạng thái hiện tại. Môi trường xác định nên hành động quyết định trạng thái kế tiếp. Môi trường rời rạc có các trạng thái và hành động riêng biệt. Chỉ có một tác tử nên không cần xét sự tương tác với tác tử khác. Môi trường tĩnh không thay đổi trong khi tác tử đang suy nghĩ.

Tác tử B nhận percept và truyền vào hàm $f_B(percept)$ để chọn hành động. Vì không cần lưu lịch sử, tác tử phản xạ đơn giản có thể phù hợp.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh

Đáp án: C.

Giải thích: Môi trường quan sát một phần đòi hỏi tác tử phải lưu thông tin từ các percept trước để suy ra trạng thái hiện tại. Tác tử C lưu danh sách percepts, sau đó gọi $f_C(percepts)$, nên có thể sử dụng lịch sử quan sát để ra quyết định.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh

Đáp án: C.

Giải thích: Môi trường vừa quan sát một phần vừa ngẫu nhiên nên tác tử cần sử dụng lịch sử percepts để ước lượng trạng thái hiện tại và có thể phải xét các kết quả xác suất. Tác tử C lưu lịch sử percepts nên phù hợp hơn A và B.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý

Đáp án: Sai.

Giải thích: Không tồn tại một tác tử duy nhất luôn hành động hợp lý trong mọi môi trường. Hành động hợp lý phụ thuộc vào percept, kiến thức, hàm đánh giá hiệu suất và môi trường cụ thể. Một tác tử có thể phù hợp trong môi trường này nhưng không phù hợp trong môi trường khác.

— Hết —